,

2026

development_system, . Telegram-: () Racket (), Racket Common Lisp. : , , Claude Code, Common Lisp, SBCL . . , , Common Lisp, .

1 3
1.1 3

2 3
2.1 3
2.2 3

3 : кнь 4
3.1 .scm 4
3.2 : dusha.rkt 4

4 : llm.rkt 5

5 : potoki.rkt 5
5.1 Claude Code 5
5.2 6
5.3 6
5.4 SBCL 7

6 : /main.rkt 8
6.1 Reply- 8
6.2 8
6.3 к нь 9
6.4 9

7 : /.lisp 9
7.1 Quicklisp 10
7.2 10
7.3 10

8 10
8.1 кнь: 11
8.2 кнь: 11

9 : kombinirovanie.rkt 12
9.1 12

10 Telegram Bot API 13
10.1 Long polling 13
10.2 13

11 14
11.1 string-prefix? Racket 14
11.2 JSON- Racket: , ... 14
11.3 : #rx vs #px 14
11.4 Quicklisp Common Lisp 14

12 15

13 : . 15

14 : 16
14.1 16
14.2 16

15 : 16

16 17
16.1 17
16.2 18

17 18

: 18

1.

. , , : , , API. development_system : .

: Common Lisp, . , SBCL. :

- ;
- , ;
- кнь () кнь ().

SICP [1]: , , , -. .

1.1.

	Racket	: , ,
	Racket + SBCL	: , ,
	Common Lisp	
Claude Code	Go ()	,

1:

2.

2.1.

. Telegram Bot API : , . к нь . /, .

```
[Telegram]  (Racket)

                Claude Code (subprocess)
                    .lisp
                SBCL (subprocess, compile-file)

                  .scm

[Telegram]  (Racket)

                SBCL (subprocess, load + )
                    OpenRouter API
```

. 1:

2.2.

. IPC, . /.scm, .

S-:

```
(
  (     "")
  (    "/path/to//.lisp")
  (   "-"))
```

1: .scm

: .

3. : кнь

3.1. .scm

, S- кнь. (system prompt), , :

```
(
  (   "
                Common Lisp.")
  (     "claude-sonnet-4-20250514")
  (    0.3)
  (     (( . "    ")))
  (      ", ,   , SICP"))
```

2: .scm

3.2. : dusha.rkt

dusha.rkt CRUD S-. -:

```
(define (-   )
  (match
    [(list '   ...)
     (for/or ([ (in-list )])
       (match
         [(list (== ) ) ]
         [_ #f]))]
    [_ #f]))
```

3: S- (dusha.rkt)

:

```
(define (-    )
  (match
    [(list '   ...)
     (cons '
           (for/list ([ (in-list )])
             (match
               [(list (== ) _) (list   )]
               [ ]))))]))
```

4:

(==) .

4. : llm.rkt

( ) OpenRouter API. llm.rkt :

```
(define (-  )
  (let* ((  (- ))
         ( (jsexpr->bytes
                (hash 'model
                      'temperature
                      'messages
                      (cons (hash 'role "system" 'content )
                            ))))
         (
          (list (string-append "Authorization: Bearer "
                               (-openrouter))
                "Content-Type: application/json"))
         ( (post-pure-port (string->url api-url) ))
         ( (bytes->jsexpr (port->bytes ))))
    (close-input-port )
    (let* (( (hash-ref 'choices #f))
           (  (and
                          (> (vector-length ) 0)
                          (vector-ref 0)))
           (  (and (hash-ref 'message #f)))
           (  (and (hash-ref 'content #f))))
      (or "  ."))))
```

5: LLM (llm.rkt)

:

1. : - "claude-sonnet-4" "anthropic/claude-sonnet-4", OpenRouter.
2. JSON Racket: racket/json JSON- , . cons, .
3. : HTTP-.

5. : potoki.rkt

, : .

5.1. Claude Code

Claude Code, API Racket:

```
(define (- )
  (let ((- (find-executable-path "claude")))
    (unless -
      (error '- " claude  PATH"))
    (let-values ([(  )
                  (subprocess #f #f #f
                              - "--print" )])
```

```
      (close-output-port )
      (let (( (port->string )))
        (close-input-port )
        (close-input-port )
        (subprocess-wait )
        (let (( (subprocess-status )))
          (if (zero? )
              (error '-
                     "Claude Code     ~a" )))))))
```

6: Claude Code (potoki.rkt)

–print Claude Code : stdin, . .

5.2.

Claude Code, кнь :

```
(define (-  )
  (let (( (or (-  ') "")))
        (     (or (-  ') "")))
    (string-append
     "     Common Lisp.\n"
     " : "  "\n"
     ": "  "\n\n"
     ": "  "\n\n"
     "  :\n"
     "-    defpackage: "
     "  (ql:quickload '(\"drakma\" \"cl-json\") :silent t)\n"
     "- HTTP: drakma:http-request; JSON: cl-json\n"
     "- Getenv: uiop:getenv (OPENROUTER_API_KEY)\n"
     "-  :-<>;  (defun  () ...)\n"
     "-   loop;   15 ;  CLOS\n"
     "-    Common Lisp\n")))
```

7:

5.3.

Claude Code markdown- (“‘lisp...“‘). - :

```
(define (- )
  (let (( (string-split  "\n")))
    (let  (( ) ( #f) ( '()))
      (cond
        [(null? )
         (if (null? )
             (string-trim )        ;
             (string-join (reverse ) "\n"))]
        [(and (not )
              (let (( (string-trim (car ))))
                (or (string-prefix?  "‘‘‘lisp")
                    (string-prefix?  "‘‘‘common-lisp"))))
         ( (cdr ) #t )]    ;
        [(and
              (string-prefix? (string-trim (car )) "‘‘‘"))
```

```
          ( (cdr ) #f )]    ;
        [
          ( (cdr ) #t (cons (car ) ))]
        [else
          ( (cdr ) #f )]))))
```

8: ( )

string-prefix? Racket. (string-prefix?), #t, . , , .

Perl- (#px):

```
(define (- )
  (let (( (regexp-match #px":-([^\\s()]+)" )))
    (if
        (string-trim (cadr ))
        "")))
```

9:

#px (PCRE) #rx ( ) : PCRE -
s
w, .

5.4. SBCL

SBCL . Quicklisp :

```
(define (- )
  (let* ((-sbcl (find-executable-path "sbcl"))
         (ql-init (path->string
                    (build-path (find-system-path 'home-dir)
                                "quicklisp" "setup.lisp")))
         (-ql (format "(when (probe-file ~s) (load ~s :verbose
             nil))"
                      ql-init ql-init))
         (- (string-append
                    "(when (find-package :ql)"
                    "  (funcall (intern \"QUICKLOAD\" :ql)"
                    "   '(\"drakma\" \"cl-json\" \"uiop\")"
                    "   :silent t))"))
         ( (format
                    "(multiple-value-bind (fasl warn fail)"
                    "    (compile-file ~s)"
                    "  (if fail (sb-ext:exit :code 1)"
                    "           (sb-ext:exit :code 0)))"
                    (path->string ))))
    (let-values ([( )
                  (subprocess #f #f #f -sbcl
                              "--noinform" "--non-interactive"
                              "--eval" -ql
                              "--eval" -
                              "--eval" )])
      (close-output-port )
      (let (( (port->string ))
            (  (port->string )))
```

```
        (close-input-port )
        (close-input-port )
        (subprocess-wait )
        (values (zero? (subprocess-status ))
                (string-append  ))))))
```

10: SBCL (potoki.rkt)

compile-file : .fasl-, . (fail) . .

6. : /main.rkt

6.1. Reply-

Telegram Reply Keyboard , . , :

```
(define (-reply )
  (hash 'text ))

(define (--reply)
  (list
   (list (-reply "  ")
         (-reply "  "))
   (list (-reply " ")
         (-reply " "))
   (list (-reply " ")
         (-reply " "))
   (list (-reply " "))))

(define (-- - )
  (- "sendMessage"
    (jsexpr->string
      (hash 'chat_id      -
            'text
            'reply_markup
            (hash 'keyboard        (--reply)
                  'resize_keyboard #t
                  'persistent      #t)))))
```

11: Reply- (main.rkt)

persistent .

6.2.

, :

```
(define (- )
  (cond
    [(or (string=? "/start")
         (string=? "/"))          #f]  ;
    [(or (string=? " ")
         (string=? "/"))         (-)]
    [(or (string=? " ")
         (string=? "/"))     (-)]
```

```
    [(or (string=? "  ")
         (string=? "/"))        (-)]
    [(or (string=? " ")
         (string=? "/"))    (-)]
    [(string-prefix? "/ ")
     (- (substring  10))]
    [else (- )]))
```

12:

6.3. к нь

к нь . -, , :

```
(define *-* (make-hash))

;;    :
(cond
  [(string=? "  ")
   (hash-set! *-* - #t)
   (- -
     "     :")]
  [(hash-ref *-* - #f)
   (hash-remove! *-* -)
   (let (( (- )))
     (-- - ))]
  [else ...])
```

13:

6.4.

. ADMIN_CHAT_ID:

```
(define (-)
  (let ((s (getenv "ADMIN_CHAT_ID")))
    (and s (string->number s))))

(define (? -)
  (let (( (-)))
    (or (not ) (= - ))))

;;    :
(when (and (? -) (eq? 'message))
  ... ; )
```

14:

, ((not) = #t). .

7. : /.lisp

Common Lisp , , . Common Lisp ( ).

7.1. Quicklisp

SBCL , Quicklisp :

```
(defun -quicklisp ()
  " Quicklisp .
  (unless (find-package :ql)
    (let ((ql-init (merge-pathnames "quicklisp/setup.lisp"
                                    (user-homedir-pathname))))
      (when (probe-file ql-init)
        (load ql-init :verbose nil :print nil))))
  (when (find-package :ql)
    (funcall (intern "QUICKLOAD" :ql)
             '("drakma" "cl-json" "flexi-streams" "uiop")
             :silent t)))
```

15: Quicklisp

intern ql:quickload : :ql .

7.2.

```
(defun - ()
  " ."
  (restart-case
      (progn
        (unless (probe-file )
          (error " : ~a" ))
        (-quicklisp)
        (load :verbose nil :print nil)
        t)
    ( ()
      :report " "
      nil)))
```

16:

7.3.

, :

```
(defun -- (-)
  " ."
  (let (( (find-package (string-upcase -))))
    (when
      (let (( (find-symbol "" )))
        (when (and (fboundp ))
          (symbol-function ))))))
```

17:

string-upcase : Common Lisp , .

8.

. , .

8.1. кнь:

iterative refinement: , .

```
(defun  (  )
  "  .   1  5."
  (if (>  5)

      (let* ((
               (if (string=  "")
                   (format nil ": ~a" )
                   (format nil
                     ": ~a~% :~%~a~%~%."
                      )))
             ( (list ()
                              ( "user" )))
             ( (- )))
        (if (- )

            (   (1+ ))))))
```

18: :

:

```
(defun - ()
  "     ."
  (and (> (length ) 20)
       (or (search "" )
           (search ":" )
           (search ":" ))))
```

19:

restart-case:

```
(defun  ()
  "    15 ."
  (restart-case
      (  "" 1)
    (- (e)
      :report "  "
      (format nil ": ~a" e))))
```

20:

8.2. кнь:

: Python- markdown- uiop:run-program.

```
(defun -python ()
  (restart-case
      (multiple-value-bind ( - )
          (uiop:run-program
           (list "python3" "-c" )
           :output          :string
           :error-output    :string
```

```
            :ignore-error-status t)
        (declare (ignore ))
        (if (and - (string/= - ""))
            (values nil -)
            (values  nil)))
    (- (e)
      (values nil (format nil ": ~A" e)))))
```

21: Python- uiop

markdown , “‘:

```
(defun - ()
  (cond ((null ) nil)
        ((- "‘‘‘" (car ))
         (- (cdr )))
        (t (cons (car )
                 (- (cdr ))))))

(defun - ( )
  (and (>= (length ) (length ))
       (string=   :end2 (length ))))
```

22: markdown-

:

```
(defun  ()
  (restart-case
      (let* ((-python (- ))
             (- (- -python)))
        (multiple-value-bind ( )
            (-python -)
          (or  )))
    ( ()
      (format nil ": ~A" ))))
```

23:

9. : kombinirovanie.rkt

, .

9.1.

-, :

```
(define (-   )
  (string-append
   "(defpackage :-" (string-join  "-") " (:use :cl))\n"
   "(in-package :-" (string-join  "-") ")\n\n"
   ;;
   (apply string-append
          (map (lambda ()
```

```
                    (format "(load ~s :verbose nil :print nil)\n" ))
                  ))
    "\n(defun  ()\n"
    "  (let  (( )\n"
    "              ( (list " (string-join
                                  (map (lambda ()
                                         (string-append "'"  ":"))
                                       )
                                  " ") ")))\n"
    "    (if (null ) \n"
    "        ( (funcall (car ) ) (cdr )))))\n"))
```

24:

10. Telegram Bot API

10.1. Long polling

long polling: getUpdates timeout=30 Telegram -.

```
(define (- )
  (let* (( (string->url
                    (string-append (-api "getUpdates")
                                   "?timeout=30&offset="
                                   (number->string ))))
          ( (get-pure-port )))
    (begin0 (port->string )
            (close-input-port ))))
```

25: Long polling Racket

offset update_id + 1 , .

10.2.

: callback-. :

```
(define (- )
  (filter-map
   (lambda ()
     (cond
       [(hash-has-key?  'message)
        (let* ((    (hash-ref  'message))
               (-  (hash-ref  'update_id 0))
               (      (hash-ref  'chat (hash)))
               (- (hash-ref  'id 0))
               (    (hash-ref  'text "")))
          (list - - 'message ))]
       [(hash-has-key?  'callback_query)
        (let* ((cb       (hash-ref  'callback_query))
               (-  (hash-ref  'update_id 0))
               (      (hash-ref
                          (hash-ref cb 'message (hash))
                          'chat (hash)))
```

```
                (- (hash-ref  'id 0))
                (cb-id   (hash-ref cb 'id ""))
                (  (hash-ref cb 'data "")))
            (list - - 'callback cb-id ))]
        [else #f]))
    ))
```

26: :

11.

:

11.1. string-prefix? Racket

(string-prefix? s1 s2) racket/string #t s2 s1. , , . Python, Haskell Go.

: (string-prefix? "/ / ") #t.

11.2. JSON- Racket: ,

racket/json JSON- Racket, . jsexpr->bytes . :

```
;;  (vector):
'messages (list->vector ...)    ;   jsexpr->bytes

;;  (list):
'messages (cons (hash 'role "system" 'content )
                )
```

27: JSON-

11.3. : #rx vs #px

Racket : #rx (POSIX-)
s,
w -. #px (PCRE):

```
;; #rx    \s:
(regexp-match #rx":-([^\s()]+)" )    ;

;; #px   :
(regexp-match #px":-([^\\s()]+)" )   ; OK
```

11.4. Quicklisp Common Lisp

ql:quickload , Quicklisp, кPackage QL does not existнь. intern:

```
;;  (     Quicklisp):
(ql:quickload '("drakma") :silent t)

;;  (      ):
(funcall (intern "QUICKLOAD" :ql) '("drakma") :silent t)
```

28: intern

12.

MASTER_BOT_TOKEN	Telegram-
APPRENTICE_BOT_TOKEN	Telegram-
OPENROUTER_API_KEY	API OpenRouter
ADMIN_CHAT_ID	Telegram ID

2:

13. : .

S-. : .

```

(
  (          "")
  (       "2026-03-13T10:17:12")
  (     "      ")
  ( ())
  (-     ())    ;   -
  (    :))   ; : | : | :
```

29:

- кнь:

```
(define (-   )
  (match
    [(list '   ...)
     (for/or ([ (in-list  )])
       (match
         [(list (==  )  )  ]
         [_ #f]))]
    [_ #f]))

;;   :
(define (-  )
  (let* ((- (build-path
                                  (string-append   ".")))
          (       (-  -)))
    (or (and   (-   '))
        "  ")))
```

30:

14. :

14.1.

with-handlers. -, JSON, API :

```
(define (-)
  (-)
  (displayln " .  ...")
  (let  (( 0))
    (with-handlers
        ([exn:fail?
          (lambda (e)
            (displayln
              (format ": ~a" (exn-message e)))
            (sleep 5)
            ( ))])   ;
      (let* ((     (- ))
             (  (-
                            (- ))))
        (let  (( ) ( ))
          (if (null? )
              ( )
              (let* (( (car ))
                     (-  (first ))
                     ...)
                ...
                ( (cdr )
                     (max  (+ - 1))))))))))
```

31:

: . , .

14.2.

, LLM :

```
(when (and (? -) (eq?  'message))
  ;;
  (let (( (fourth )))
    ...))
;;
```

32:

15. :

: Telegram SBCL-.

```
(define (--sbcl - - )
  (let* ((-sbcl (find-executable-path "sbcl"))
         (        (->string (-)))
```

```
          (
           (format "(load ~s :verbose nil :print nil)" ))
          (
           (format
            "(let (( (funcall (intern \"\" \"\")
                                ~s ~s ~s)))
               (princ ) (finish-output)
               (sb-ext:exit :code 0))"
            (->string -) - )))
     (let-values ([(    )
                   (subprocess #f #f #f
                               -sbcl
                               "--noinform" "--non-interactive"
                               "--eval"
                               "--eval" )])
       (close-output-port )
       (let (( (port->string ))
             (     (port->string )))
         (close-input-port )
         (close-input-port )
         (subprocess-wait )
         (if (zero? (subprocess-status ))
             (string-trim )
             (format " SBCL: ~a" (string-trim )))))))
```

33: : Racket-

, :

```
(defun  (- - )
  " ,   , ."
  (- -)
  (let ((- (-- -)))
    (unless -
      (error "      ~a"
             -))
    (-- - )))

(defun -- ( )
  "      ."
  (restart-case
      (funcall  )
    (- ()
      :report "    "
      )))
```

34: (CL)

16.

16.1.

:

(long poll)	<100	
SBCL	1–3	
Quicklisp +	2–5	drakma, cl-json
OpenRouter API	1–5	
( )	4–13	
(Claude Code)	15–60	

3:

SBCL: . SBCL- (save-lisp-and-die).

16.2.

1. SBCL: 4–8 <100 .
2. -: SBCL SWANK- .
3. : .fasl drakma .

17.

---

development_system, . :

1. : Common Lisp .
2. : (Racket,) (SBCL,) .
3. : IPC .
4. Common Lisp: restart-case .
5. SICP: , , - Racket .

: , , ACL2.

:

```
development_system/
 /
    main.rkt            # :  , ,
    dusha.rkt           # : / S-
    potoki.rkt          #
    kombinirovanie.rkt # : , ,
    llm.rkt             #  OpenRouter API
    .scm            #
    /            #  ( , .txt)
    /
        .scm   #
        .lisp # :
        . #
        .lisp # :
```

```
        .

/
   main.rkt            # -:   ,
   .lisp    # CL-:  ,  , condition/restart

docs/
    plans/
       TODO.md         #
    paper.tex           #
```

35: development_system

( ):

/main.rkt	∼330	Racket
/potoki.rkt	∼270	Racket
/dusha.rkt	∼120	Racket
/kombinirovanie.rkt	∼160	Racket
/llm.rkt	∼40	Racket
/main.rkt	∼260	Racket
/.lisp	∼100	Common Lisp
	∼1280	

4:

[1] Abelson, H., Sussman, G.J. Structure and Interpretation of Computer Programs. MIT Press, 1996.

[2] SBCL Team. SBCL User Manual. https://www.sbcl.org/manual/

[3] Flatt, M., PLT. Reference: Racket. https://docs.racket-lang.org/reference/

[4] OpenRouter. OpenRouter API Documentation. https://openrouter.ai/docs

[5] Telegram. Bot API Reference. https://core.telegram.org/bots/api